

TD n°2 : Constraint Solving Problems

Ce TD aborde le type de traitement mis en œuvre dans la résolution de *problèmes de satisfaction de contraintes* (ou *CSP*), problèmes classiquement associés à une exploration combinatoire d'un espace de solutions. Une illustration en est proposée par le biais de la résolution d'une grille de Sudoku.

Backtracking

L'algorithme le plus simple pour résoudre un problème de satisfaction de contraintes est le *backtracking* ; c'est un algorithme dit "brute force" qui sélectionne l'ordre dans lequel instancier les variables, puis l'ordre dans lequel essayer les valeurs affectées récursivement aux variables. Chaque fois qu'une variable est instanciée, toutes les contraintes qui la contiennent et qui ne contiennent que des variables instanciées sont testées : si une contrainte n'est pas vérifiée, l'algorithme arrête sa recherche et essaie la valeur suivante pour la variable (ce qui correspond à un retour en arrière dans l'exploration de l'arbre des affectations possibles, d'où le terme *backtrack*). Un ensemble de variables instanciées est *consistant* s'il vérifie toutes les contraintes associées. Si un sous-ensemble de l'ensemble des contraintes n'est pas consistant, alors aucun ensemble de variables instanciées qui le contient ne peut être consistant, raison pour laquelle on arrête la recherche dès qu'une contrainte n'est pas vérifiée. L'arbre des affectations est exploré en *profondeur d'abord* (en opposition à *largeur d'abord*, afin de minimiser la mémoire consommée et parce qu'on connaît à l'avance la profondeur de recherche du problème. Avec A , liste des affectations (initialement vide) et V liste des variables (initialement contenant toutes les variables du problème), l'algorithme est :

```
Backtrack( $A, V$ )  
si ( $V == \emptyset$ ) alors  
  |  $A$  est une solution  
sinon  
  | choisir  $x \in V$   
  |  $d \leftarrow d_x$   
  | tant que ( $d \neq \emptyset$ ) faire  
  |   | choisir  $v$  dans  $d$   
  |   |  $d \leftarrow d \setminus \{v\}$   
  |   | si ( $A \cup \{x \leftarrow v\}$  est consistante) alors  
  |   |   | Backtrack( $A \cup \{x \leftarrow v\}, V \setminus \{x\}$ )  
  |   |   fin  
  |   fin  
fin
```

Sudoku avec backtracking simple

Un problème de Sudoku est une matrice 9×9 telle que chaque ligne et chaque colonne ne contient qu'une seule fois les entiers de 1 à 9. Une grille 9×9 est décomposée en sous-matrices 3×3 . En plus des contraintes sur les lignes et les colonnes, chaque carré 3×3 ne doit contenir qu'une seule fois les chiffres 1 à 9. Enfin, il n'y a (évidemment) que neuf valeurs possibles pour chaque entrée de la matrice.

Exercice 1 A partir du formalisme introduit en cours, modéliser le problème de Sudoku comme un *CSP*.

Exercice 2 A partir de l'algorithme backtrack ci-dessus, écrire la fonction python *Backtrack* qui permet l'exploration récursive d'un arbre d'affectation, appliqué à la représentation d'une grille de Sudoku codée sous forme d'une liste de listes. Résoudre la grille :

4			3		7	6
	9	3		7		
2			1	4		
9	5	8				7
3	4				7	
	7	2	8	9	3	
	8		2			3
		4				7
				5		4
					6	
						1
						8
						3
						5
						9
						6

Sudoku avec Forward Checking

Dans l'algorithme de *Forward Checking*, après chaque affectation d'une valeur à une variable réalisée au cours du backtrack, on retire des domaines des variables restantes les valeurs qui ne sont pas compatibles avec l'affectation en cours. Cette mise à jour des domaines à chaque affectation d'une variable permet de détecter les domaines contraints et les domaines vides. Avec A , liste des affectations (initialement vide) et V liste des variables (initialement contenant toutes les variables du problème), l'algorithme *Forward Checking* obtenu à partir de l'algorithme *backtrack* est :

```
Forward-Checking( $A, V$ )
si ( $V == \emptyset$ ) alors
  |  $A$  est une solution
sinon
  | Choisir  $x \in V$ 
  |  $d \leftarrow d_x$ 
  | tant que ( $d \neq \emptyset$ ) faire
  |   | Choisir  $v$  dans  $d$ 
  |   |  $d \leftarrow d \setminus \{v\}$ 
  |   | Sauvegarde( $V \setminus \{x\}$ )
  |   | si Check-Forward( $x, v, V$ ) alors
  |   |   | Forward-Checking( $A \cup \{x \leftarrow v\}, V \setminus \{x\}$ )
  |   |   fin
  |   | Restauration( $V \setminus \{x\}$ )
  |   fin
  fin
fin

Check-Forward( $x_i, v, V$ )
 $Consistant \leftarrow \text{True}$ 
pour chaque  $x_j \in V \setminus \{x_i\}$  faire
  | tant que  $Consistant$  faire
  |   | pour chaque  $v' \in d_j$  faire
  |   |   | si  $x_i \leftarrow v, x_j \leftarrow v'$  est non-consistante alors
  |   |   |   |  $d_j \leftarrow d_j \setminus \{v'\}$ 
  |   |   |   fin
  |   |   fin
  |   | si ( $d_j == \emptyset$ ) alors
  |   |   |  $Consistant \leftarrow \text{False}$ 
  |   |   fin
  |   fin
  fin
  Retourner  $Consistant$ 
fin
```

Exercice 3 Modifier le programme de résolution de Sudoku par backtrack en y intégrant le Forward Checking.

Heuristiques de choix de variables

Lorsqu'on explore un arbre entièrement, l'ordre d'instanciation des variables importe peu mais si, comme dans le backtracking, on permet de couper l'arbre dès qu'une contrainte n'est plus vérifiée, il est alors préférable de couper le plus tôt possible une branche qui ne conduira pas à une solution. L'idée est donc d'instancier en priorité les variables qui amènent à violer une contrainte ; cette heuristique s'appelle *le principe de l'échec*

d'abord. L'ordre d'instanciation des variables peut être décidé de manière statique une fois pour toute avant l'exécution, ou dynamiquement lors de l'exécution. Un exemple de bonne heuristique statique consiste à choisir à chaque instanciation la variable qui est liée par des contraintes au plus grand nombre de variables déjà choisies (heuristique de *cardinalité maximum*). D'autres heuristiques consistent à prendre en compte la taille du domaine de la variable, le nombre de contraintes dans lesquelles la variable est présente ou encore la difficulté des contraintes contenant la variable. Un exemple de bonne heuristique dynamique est de choisir la variable ayant le nombre minimal de valeurs vérifiant les contraintes étant données les instanciations déjà effectuées.

Exercice 4 L'ordre d'instanciation des valeurs n'a pas d'influence sur les problèmes inconsistants. Pourquoi ?

Exercice 5 Modifier le programme de Sudoku pour qu'il commence par instancier les variables de plus petit domaine.